DOCUMENT INTERNE — CONFIDENTIEL

HOK REPORTS

Documentation Technique Complète

VERSION 2.0.0 MARS 2026 CABINET HOK

TABLE DES MATIÈRES

01 Présentation & Stack technique

02 Architecture & Structure du projet

03 Schéma de la base de données (10 tables)

04 Diagramme relationnel & Index

05 Routes API — Référence complète (40 endpoints)

06 Librairies internes (src/lib)

07 Composants & Pages

08 Sécurité & Middleware

09 Moteur IA (Groq) & WhatsApp

10 Internationalisation & Journal d'audit

SECTION 01

Présentation & Stack technique

Vue d'ensemble de la plateforme et de ses technologies.

HOK Reports est une plateforme interne de gestion documentaire pour le Cabinet HOK (Avocats — Cotonou, Bénin). Elle automatise la transcription audio, la génération de rapports juridiques par IA, le suivi des audiences et la gestion des dossiers.

> **Mission** — Automatiser la transcription, la synthèse et le suivi des audiences pour permettre aux avocats de se concentrer sur leur métier.

COUCHE	TECHNOLOGIE
Framework	Next.js 16.1.6 (App Router, Turbopack)
Langage	TypeScript 5.x
Styles	Tailwind CSS 4.x
Base de données	PostgreSQL via Prisma v7 + `@prisma/adapter-pg`
Auth	NextAuth v5 (beta.30) — JWT + Credentials + Google OAuth
État client	Zustand 5.x (persistance localStorage)
IA / LLM	Groq uniquement — Whisper `whisper-large-v3-turbo` (transcription) + `llama-3.3-70b-versatile` (rapports & parsing WhatsApp)
Icônes	Lucide React 0.574
PDF	`@react-pdf/renderer` + `pdf-lib`
i18n	Système personnalisé (fr / en / fon) — ~370 clés
Hachage	bcryptjs (salt factor 12)

CLÉ API REQUISE

GROQ_API_KEY

Format `gsk_...` — configurable en DB ou .env

CLIENT PRISMA

v7 + PG Pool

Import depuis `@/generated/prisma/client`, adaptateur `@prisma/adapter-pg`

SECTION 02

Architecture & Structure du projet

Organisation du code, flux de données et architecture d'authentification.

```
hok-reports/

├── prisma/schema.prisma      10 modèles, PostgreSQL

├── prisma.config.ts          Datasource URL

├── src/

│   ├── app/

│   │   ├── (dashboard)/        admin, dashboard, sessions, flash, reports, roles, profile, settings, about,
documentation

│   │   ├── api/                40 endpoints (19 fichiers route.ts)

│   │   ├── login/, register/  Pages publiques

│   │   ├── platform-docs/     Doc protégée par mot de passe

│   │   └── mentions-legales/, confidentialite/, cgu/

│   ├── components/

│   │   ├── layout/            sidebar.tsx, header.tsx, dashboard-shell.tsx

│   │   └── ui/                Composants réutilisables

│   ├── lib/                   auth, prisma, llm, audit, sanitize, rate-limit, whatsapp-parser, whatsapp, i18n, store,
utils

│   ├── generated/prisma/      Client Prisma généré

│   └── middleware.ts          Auth + headers sécurité

├── docs/                      Documentation HTML

└── data/audio/                Fichiers audio (hors git)
```

Flux de données

> **Navigateur** → Middleware (NextAuth + Headers) → API Route (Auth + Sanitize + Rate Limit) → **Prisma ORM** → PostgreSQL

Architecture d'authentification (split edge)

FICHIER	RÔLE
`auth.config.ts`	Config edge-compatible (middleware). Callbacks jwt/session/authorized. Définit pages publiques, redirections admin, protection des routes.
`auth.ts`	Config complète : Credentials (email + bcrypt) + Google OAuth. Gère signIn Google (création compte `pending`), mise à jour `lastLoginAt`, audit log, rate limit login (5/5min).
`middleware.ts`	Utilise auth.config.ts. Headers sécurité (X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Referrer-Policy, Permissions-Policy). Bloque `/api/admin/*` pour non-admins.

Client Prisma v7

Instancié via `@prisma/adapter-pg` avec pool de connexions `pg`. Détection localhost pour désactivation SSL. Pattern singleton global en dev.

```
import { PrismaClient } from "@/generated/prisma/client";
import { PrismaPg } from "@prisma/adapter-pg";
import { Pool } from "pg";
const pool = new Pool({ /* config depuis DATABASE_URL */ });
const adapter = new PrismaPg(pool);
const prisma = new PrismaClient({ adapter });
```

SECTION 03

Schéma de la base de données

10 tables — description exhaustive de chaque colonne, type, contrainte et relation.

3.1 — User

Table centrale. Toutes les entités principales sont liées via `onDelete: Cascade`.

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`id`	String	PK cuid()	Identifiant unique
`name`	String	Requis	Nom complet
`email`	String	UNIQUE	Email (login)
`password`	String?	NULL	Hash bcrypt. `null` = Google-only
`role`	String	Déf: "user"	`admin` | `user`
`status`	String	Déf: "pending"	`pending` | `active` | `rejected`
`blocked`	Boolean	Déf: false	Bloqué par admin
`avatar`	String?	NULL	URL avatar Google
`language`	String	Déf: "fr"	fr | en | fon
`lastLoginAt`	DateTime?	NULL	Dernière connexion
`createdAt`	DateTime	now()	Création
`updatedAt`	DateTime	@updatedAt	Mise à jour

Relations : sessions[], reports[], loginHistory[], auditLogs[], hearingReports[], weeklyRoles[]

3.2 — Session

Session d'enregistrement audio avec cycle de vie à 5 états.

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`id`	String	PK	Identifiant
`title`	String	Requis, max 200	Titre

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`description`	String?	max 2000	Description
`clientName`	String	Requis, max 100	Client
`clientEmail`	String?		Email client
`clientPhone`	String?		Téléphone
`caseReference`	String?		Réf dossier
`audioUrl`	String?		Chemin audio
`audioDuration`	Int?		Durée (sec)
`status`	String	Déf: "recording"	recording | transcribing | summarizing | completed | error
`language`	String	Déf: "fr"	fr | en
`userId`	String	FK → User	Propriétaire

Relations : user, transcription? (1:1), reports[], notes[] — **Index :** userId, createdAt — **Cascade :** depuis User

---

3.3 — Transcription

Transcription Whisper. Relation **1:1** avec Session.

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`id`	String	PK	Identifiant
`content`	String	Requis	Texte complet
`segments`	String?	JSON	Segments horodatés [{start, end, text}]
`language`	String	Déf: "fr"	Langue détectée
`confidence`	Float?		Score confiance
`sessionId`	String	FK UNIQUE	Session (1:1)

3.4 — Report

Rapport juridique généré par IA. Contient synthèse, points clés, actions, observations juridiques et suggestions typées.

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`id`	String	PK	Identifiant
`title`	String	Requis	Titre IA
`summary`	String	Requis	Synthèse narrative
`keyPoints`	String	JSON	["Point 1","Point 2"]
`actionItems`	String?	JSON	["Action 1","Action 2"]
`legalNotes`	String?		Observations juridiques
`suggestions`	String?	JSON	["[LOI] ...","[ARGUMENT] ...","[DEFENSE] ...","[OUVERTURE] ...","[JURISPRUDENCE] ...","[PREUVE] ..."]
`category`	String	Déf: "general"	general | consultation | hearing | deposition | meeting
`status`	String	Déf: "draft"	draft | final | archived
`format`	String	Déf: "standard"	standard | detailed | brief
`exportedAt`	DateTime?		Dernier export PDF
`sessionId`	String	FK	Session source
`userId`	String	FK	Auteur

Index : userId, sessionId, category, createdAt

3.5 — Note

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`id`	String	PK	Identifiant
`content`	String	max 5000	Contenu
`timestamp`	Int?		Timestamp audio (sec)
`isImportant`	Boolean	Déf: false	Marqué important
`sessionId`	String	FK	Session parente

Index : sessionId

Schéma DB (suite)

3.6 — HearingReport

Compte rendu d'audience. Source : manuelle ou WhatsApp.

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`id`	String	PK	Identifiant
`hearingDate`	DateTime	Requis	Date audience
`jurisdiction`	String	Requis	Tribunal
`chamber`	String?		Chambre
`caseReference`	String	Requis	Réf dossier
`clientName`	String	Requis	Client
`opponent`	String?		Adversaire
`lawyerName`	String?		Avocat
`outcome`	String	Requis	Compte rendu
`nextHearingDate`	DateTime?		Prochaine audience
`tasks`	String?	JSON	Tâches ["..."]
`notes`	String?		Notes
`status`	String	Déf: "active"	active | archived
`source`	String	Déf: "manual"	manual | whatsapp
`whatsappMessageId`	String?		ID message WhatsApp
`whatsappSender`	String?		Expéditeur WhatsApp
`userId`	String	FK	Utilisateur

Index : userId, hearingDate, nextHearingDate, caseReference, createdAt, source

---

3.7 — WeeklyRole

Rôle d'audience hebdomadaire auto-généré depuis les HearingReport.

COLONNE	TYPE	CONTRAINTES	DESCRIPTION
`id`	String	PK	Identifiant
`weekStart`	DateTime	Requis	Lundi
`weekEnd`	DateTime	Requis	Dimanche
`title`	String	Requis	Titre auto-généré
`entries`	String	JSON	[{hearingReportId, hearingDate, jurisdiction, caseReference, clientName, opponent, lawyerName, previousOutcome, tasks[], notes}]
`notes`	String?		Notes
`status`	String	Déf: "draft"	draft | published | archived
`userId`	String	FK	Propriétaire

Unique : UNIQUE (weekStart, userId) — **Index :** userId, weekStart, status

---

3.8 — LoginHistory

COLONNE	TYPE	DESCRIPTION
`id`	String PK	Identifiant
`ipAddress`	String?	IP client
`userAgent`	String?	Navigateur
`success`	Boolean (déf: true)	Connexion réussie
`userId`	String FK	Utilisateur
`createdAt`	DateTime	Date tentative

3.9 — SystemSetting

COLONNE	TYPE	DESCRIPTION
`id`	String PK	Identifiant
`key`	String UNIQUE	Clé (ex: "groq_api_key")
`value`	String	Valeur

3.10 — AuditLog

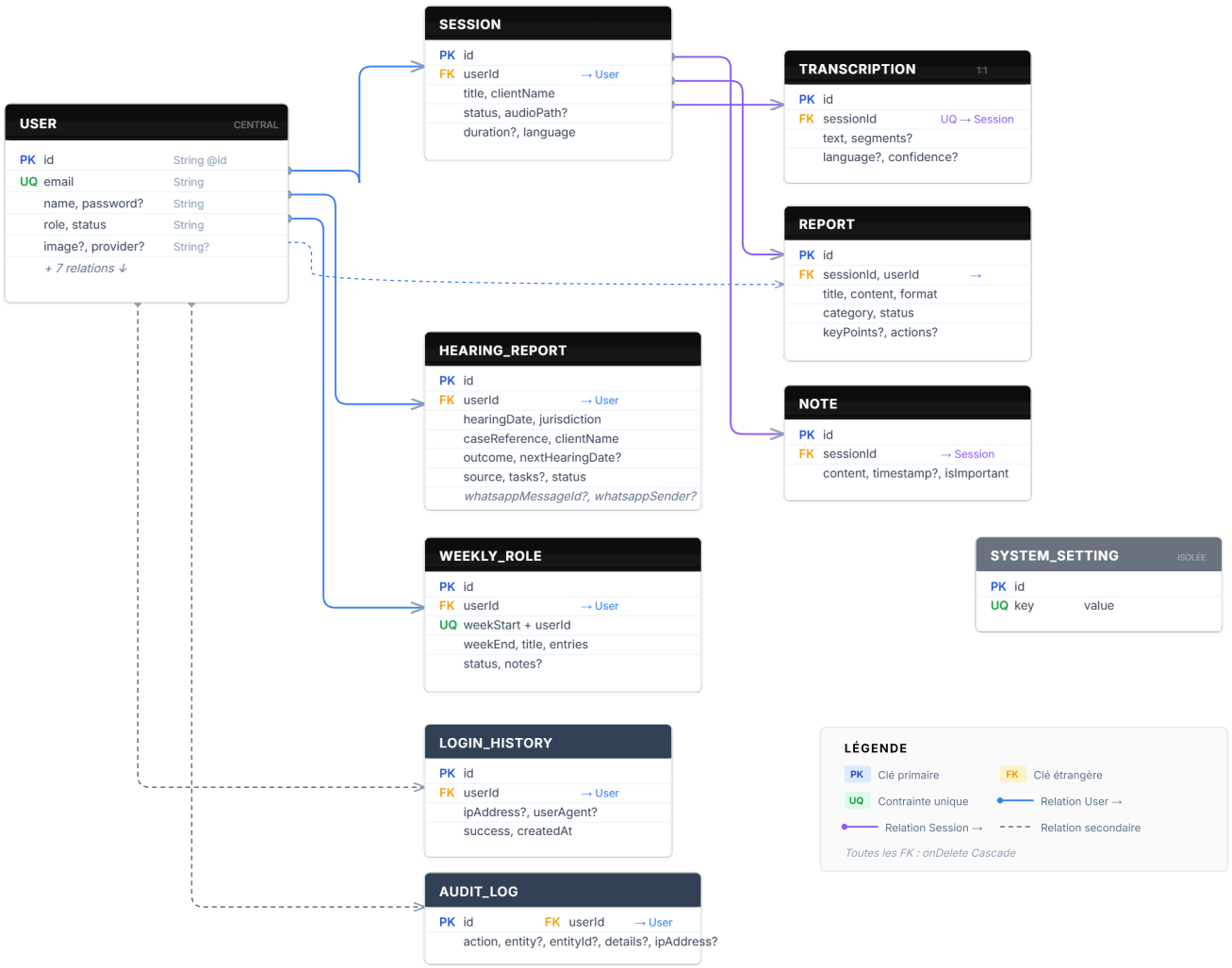
COLONNE	TYPE	DESCRIPTION
`id`	String PK	Identifiant
`action`	String	Type d'action (voir section 10)
`entity`	String?	session | report | user | settings | hearing_report | weekly_role
`entityId`	String?	ID entité
`details`	String? (JSON)	Détails variables
`ipAddress`	String?	IP client
`userAgent`	String?	Navigateur
`userId`	String FK	Auteur action
`createdAt`	DateTime	Horodatage

Index : userId, action, createdAt

SECTION 04

Diagramme relationnel de la base de données

Vue complète des 10 tables, colonnes clés, relations et règles de cascade. Toutes les FK utilisent `onDelete: Cascade`.



Inventaire des index

TABLE	COLONNES	TYPE
User	email	UNIQUE
Session	userId, createdAt	INDEX
Transcription	sessionId	UNIQUE
Report	userId, sessionId, category, createdAt	INDEX
Note	sessionId	INDEX

TABLE	COLONNES	TYPE
HearingReport	userId, hearingDate, nextHearingDate, caseReference, createdAt, source	INDEX
WeeklyRole	(weekStart, userId)	UNIQUE
WeeklyRole	userId, weekStart, status	INDEX
LoginHistory	userId, createdAt	INDEX
SystemSetting	key	UNIQUE
AuditLog	userId, action, createdAt	INDEX

SECTION 05

Routes API — Référence complète

40 endpoints REST organisés par domaine.

5.1 — Authentification & Inscription

VERBE	ROUTE	AUTH	DESCRIPTION
*	/api/auth/[...nextauth]	Non	Endpoints NextAuth v5 (signIn, signOut, session, callback)
POST	/api/register	Non	Inscription. Validation nom/email/mdp (8+ car, maj, min, chiffre, spécial), hash bcrypt (12), status "pending". Rate: 5/5min/IP

5.2 — Dashboard & Health

VERBE	ROUTE	AUTH	DESCRIPTION
GET	/api/dashboard	Oui	Stats: totalSessions, totalReports, totalDuration, 5 récentes. Admin=tout, user=ses données.
GET	/api/health	Non	Health check (user count, timestamp)
POST	/api/docs	Non	Accès doc HTML protégée par mot de passe. Rate: 5/15min/IP

5.3 — Sessions

VERBE	ROUTE	AUTH	DESCRIPTION
GET	/api/sessions	Oui	Liste paginée (search, status, page, limit max 100)
POST	/api/sessions	Oui	Création. Requis: title, clientName. Sanitisation. Audit.
GET	/api/sessions/[id]	Oui	Détail + transcription + rapports + notes. Vérif ownership.
PATCH	/api/sessions/[id]	Oui	Modification (whitelist champs). Audit.
DEL	/api/sessions/[id]	Oui	Suppression + fichier audio disque. Audit.

5.4 — Audio, Transcription & Flash

VERBE	ROUTE	AUTH	DESCRIPTION
POST	/api/upload	Oui	Upload audio (MP3,WAV,WebM,OGG,MP4,M4A). Max 100Mo. Rate: 20/min
POST	/api/transcribe	Oui	Transcription Groq Whisper. Rate: 10/min
GET	/api/audio/[filename]	Oui	Sert le fichier audio

VERBE	ROUTE	AUTH	DESCRIPTION
GET	/api/transcriptions/[id]	Oui	Récupère transcription par session
POST	/api/flash	Oui	Flash 4 étapes: Transcribe → Extract metadata → Update session → Generate report. Rate: 5/min

5.5 — Rapports

VERBE	ROUTE	AUTH	DESCRIPTION
GET	/api/reports	Oui	Liste paginée (search, category, status, page, limit)
POST	/api/reports	Oui	Génération rapport IA (Groq llama-3.3-70b). Params: sessionId, format, category
GET	/api/reports/[id]	Oui	Détail + session + transcription
PATCH	/api/reports/[id]	Oui	Modification (whitelist). Audit.
DEL	/api/reports/[id]	Oui	Suppression. Audit.
GET	/api/reports/[id]/pdf	Oui	Export PDF. En-tête Cabinet HOK, points clés, suggestions typées, transcription

5.6 — Notes & Profil

VERBE	ROUTE	AUTH	DESCRIPTION
POST	/api/notes	Oui	Créer note (sessionId, content max 5000, timestamp, isImportant)
DEL	/api/notes?id=xxx	Oui	Supprimer note
GET	/api/profile	Oui	Infos profil
PATCH	/api/profile	Oui	Modifier nom. Audit.
POST	/api/profile/password	Oui	Changer mdp (vérif ancien, validation nouveau, hash bcrypt). Audit.

Routes API (suite)

5.7 — Comptes rendus d'audience

VERBE	ROUTE	AUTH	DESCRIPTION
GET	/api/hearing-reports	Oui	Liste paginée (search, status)
POST	/api/hearing-reports	Oui	Création. Requis: hearingDate, jurisdiction, caseReference, clientName, outcome. Audit.
GET	/api/hearing-reports/[id]	Oui	Détail
PATCH	/api/hearing-reports/[id]	Oui	Modification. Audit.
DEL	/api/hearing-reports/[id]	Oui	Suppression. Audit.

5.8 — Rôles hebdomadaires

VERBE	ROUTE	AUTH	DESCRIPTION
GET	/api/weekly-roles	Oui	Liste paginée (status)
POST	/api/weekly-roles	Oui	Générer/régénérer rôle. Agrège HearingReport actifs de la semaine. Upsert. Audit.
GET	/api/weekly-roles/[id]	Oui	Détail
PATCH	/api/weekly-roles/[id]	Oui	Publier/archiver. Audit.
DEL	/api/weekly-roles/[id]	Oui	Supprimer. Audit.

5.9 — Paramètres & WhatsApp

VERBE	ROUTE	AUTH	DESCRIPTION
GET	/api/settings/engine-status	Oui	Vérifie si Groq est configuré
POST	/api/settings/engine-key	Admin	Sauver clé Groq (préfixe gsk_)
GET	/api/whatsapp/webhook	Non	Vérification webhook Meta
POST	/api/whatsapp/webhook	Non	Réception messages. Détection IA (≥0.6) + extraction IA + création HearingReport
GET	/api/whatsapp/status	Admin	Statut connexion WhatsApp
POST	/api/whatsapp/test	Admin	Simuler message WhatsApp

5.10 — Administration (toutes protégées admin)

VERBE	ROUTE	DESCRIPTION
GET	/api/admin/users	Liste utilisateurs + compteurs
PATCH	/api/admin/users	Changer rôle. Anti auto-changement. Audit.
POST	/api/admin/users/approve	Approuver/rejeter. Audit.
POST	/api/admin/users/block	Bloquer/débloquer. Audit.
POST	/api/admin/users/edit	Éditer nom/email. Vérif doublons. Audit.
DEL	/api/admin/users/delete	Supprimer (cascade). Anti auto-suppression. Audit.
POST	/api/admin/users/reset-password	Reset mdp. Validation complexité, anti self-reset, anti Google-only, bcrypt. Rate: 10/10min. Audit.
GET	/api/admin/stats	Stats globales (users, sessions, rapports, durée)
GET	/api/admin/activity	Activité récente
GET	/api/admin/audit	Journal d'audit complet
GET	/api/admin/reports	Rapports admin
GET	/api/admin/sessions	Sessions admin
GET	/api/admin/llm	Config LLM (clé masquée)
POST	/api/admin/llm	Sauver clé Groq (gsk_). Audit.
POST	/api/admin/llm/test	Tester clé Groq via /v1/models

SECTION 06

Librairies internes (src/lib)

Détail de chaque module utilitaire du projet.

6.1 — llm.ts (Moteur IA Groq)

Fichier central de l'intelligence artificielle. Utilise exclusivement **Groq** (clé format `gsk_...`).

- `getGroqApiKey()` — Récupère la clé (DB prioritaire, puis .env)
- `validateGroqKey(key)` — Vérifie le préfixe `gsk_`
- `getGroqTranscribeClient()` — Client OpenAI SDK pointé vers `api.groq.com`
- `transcribeAudio(buffer, lang)` — Whisper `whisper-large-v3-turbo` → {text, segments}
- `extractSessionMetadata(text)` — IA extrait titre, client, référence, catégorie, description
- `generateReport(text, client, ref, format, lang)` — Génère rapport juridique complet via `llama-3.3-70b-versatile`
- `generateJSON(prompt, schema)` — Génération JSON structuré avec validation

6.2 — sanitize.ts

- `sanitizeString(str)` — Strip HTML, scripts, iframes, event handlers, entités dangereuses
- `validateEmail(email)` — Regex standard
- `validatePassword(pwd)` — 8+ car, majuscule, minuscule, chiffre, spécial, max 128
- `validateName(name)` — 2-100 car, pas de caractères spéciaux dangereux
- `filterSessionFields(body)` / `filterReportFields(body)` — Whitelist des champs modifiables

6.3 — audit.ts

- `logAudit({userId, action, entity?, entityId?, details?, ipAddress?, userAgent?})` — Enregistre dans AuditLog
- `getClientInfo(req)` — Extrait IP (x-forwarded-for / x-real-ip) et User-Agent

6.4 — rate-limit.ts

- `rateLimit(key, maxRequests, windowMs)` — Limiter en mémoire (Map). Fenêtre glissante. Retourne `{allowed, remaining}`
- Nettoyage automatique des entrées expirées toutes les 5 minutes via `setInterval`

6.5 — whatsapp-parser.ts & whatsapp.ts

- `detectHearingReportAI(text)` — IA détecte si un message contient un compte rendu (confiance ≥ 0.6)
- `parseWhatsAppMessageAI(text, sender?)` — IA extrait données structurées + résumé professionnel juridique
- `isHearingReportKeywords(text)` / `parseWhatsAppMessageRegex(text)` — Fallback regex
- `isWhatsAppConfigured()` / `getVerifyToken()` / `sendReportConfirmation()` / `sendParseErrorReply()`

6.6 — Autres modules

FICHIER	RÔLE
`store.ts`	Zustand store : locale (fr/en/fon), sidebarOpen. Persisté localStorage (`hok-app-store`)
`i18n.ts`	~370 clés de traduction. Fonction `t(key, locale)` . Fallback français. Type `Locale = "fr"|"en"|"fon"`
`utils.ts`	Fonction `cn()` = clsx + tailwind-merge pour les classes conditionnelles
`prisma.ts`	Client Prisma v7 singleton avec Pool PG et détection SSL automatique
`auth.ts`	NextAuth complet (Credentials + Google OAuth + callbacks)
`auth.config.ts`	Config edge-compatible pour middleware (pas d'imports Node)

SECTION 07

Composants & Pages

Navigation et structure de l'interface utilisateur.

Navigation principale (sidebar)

ROUTE	LABEL	ACCÈS	DESCRIPTION
`/dashboard`	Tableau de bord	Tous	Stats, sessions et rapports récents
`/sessions`	Sessions	Tous	Liste, recherche, filtrage par statut
`/sessions/new`	Nouvelle session	Tous	Création + enregistrement audio navigateur (MediaRecorder)
`/sessions/[id]`	Détail session	Tous	Lecteur audio, transcription, notes horodatées, rapports
`/flash`	Flash Record	Tous	Enregistrement 1 clic, transcription + rapport auto. Format WebM/Opus.
`/reports`	Rapports	Tous	Liste, recherche, filtrage catégorie/statut
`/reports/[id]`	Détail rapport	Tous	Synthèse, points clés, actions, suggestions IA, export PDF
`/roles`	Rôles d'audience	Tous	Comptes rendus d'audience, rôles hebdomadaires, génération, publication

Navigation secondaire

ROUTE	LABEL	ACCÈS	DESCRIPTION
`/admin`	Administration	Admin	8 onglets: overview, pending, users, reports, sessions, activity, audit, llm
`/profile`	Mon profil	Tous	Nom, email, changement mot de passe
`/settings`	Paramètres	Tous	Langue, config WhatsApp, clé IA
`/documentation`	Documentation	Tous	Lien vers la documentation
`/about`	À propos	Tous	Informations sur la plateforme

Pages publiques

ROUTE	DESCRIPTION
`/login`	Connexion (email/mdp + Google OAuth)
`/register`	Inscription (nom, email, mdp avec règles de complexité)
`/platform-docs`	Documentation protégée par mot de passe (DOCS_PASSWORD)
`/mentions-legales`	Mentions légales

ROUTE	DESCRIPTION
`/confidentialite`	Politique de confidentialité
`/cgu`	Conditions générales d'utilisation

Composants layout

- `sidebar.tsx` — Sidebar rétractable (60px/240px). Liens dynamiques selon le rôle. Logo HOK. Version en pied.
- `header.tsx` — Barre supérieure avec titre de page, sélecteur langue et bouton déconnexion
- `dashboard-shell.tsx` — Shell qui combine sidebar + header + contenu principal

SECTION 08

Sécurité & Middleware

Toutes les mesures de protection implémentées.

Authentification

- **JWT via NextAuth v5**
  Stratégie de session JWT signée avec AUTH_SECRET. Token contient id, role, picture.

- **Hachage bcrypt (salt 12)**
  Mots de passe hachés avant stockage. Comparaison sécurisée via `bcrypt.compare()`.

- **Google OAuth**
  Authentification complémentaire. Création auto de compte en `pending`. Linking d'email autorisé.

Autorisation

- **Vérification d'ownership (anti-IDOR)**
  Chaque route API vérifie que l'utilisateur est propriétaire de la ressource. Les admins peuvent accéder à toutes les ressources.

- **Rôles différenciés**
  Admin : accès complet. User : uniquement ses propres données. Vérification double (middleware + route).

- **Approbation d'inscription**
  Nouveaux comptes en `pending`. Activation manuelle par admin. Comptes bloqués/rejetés ne peuvent pas se connecter.

Validation des entrées

- **Sanitisation XSS**
  Nettoyage systématique : HTML entities, scripts, iframes, event handlers (`sanitizeString`).

- **Validation stricte**
  Email regex, mot de passe (8+ car, maj, min, chiffre, spécial, max 128), nom (2-100 car).

- **Whitelist de champs**
  Seuls les champs autorisés sont acceptés lors des PATCH (`filterSessionFields`, `filterReportFields`).

Headers HTTP de sécurité (middleware)

HEADER	VALEUR	PROTECTION
`X-Content-Type-Options`	nosniff	MIME sniffing
`X-Frame-Options`	DENY	Clickjacking
`X-XSS-Protection`	1; mode=block	XSS navigateur
`Referrer-Policy`	strict-origin-when-cross-origin	Fuite referer
`Permissions-Policy`	camera=(), microphone=(self), geolocation=()	APIs navigateur

Rate Limiting

ENDPOINT	LIMITE	FENÊTRE	CLÉ
Login	5 tentatives	5 min	email
Inscription	5 tentatives	5 min	IP
Upload audio	20 requêtes	1 min	userId
Transcription	10 requêtes	1 min	userId
Flash	5 requêtes	1 min	userId
Admin reset password	10 tentatives	10 min	adminId
Accès docs	5 tentatives	15 min	IP

SECTION 09

Moteur IA (Groq) & Intégration WhatsApp

Architecture IA et pipeline WhatsApp.

9.1 — Moteur IA Groq

La plateforme utilise **exclusivement Groq** comme fournisseur IA. Aucune dépendance OpenAI ou Anthropic directe.

TRANSCRIPTION

Whisper

`whisper-large-v3-turbo` via Groq API

GÉNÉRATION

LLaMA 3.3

`llama-3.3-70b-versatile` pour rapports, métadonnées, parsing WhatsApp

La clé API Groq (`gsk_...`) peut être configurée via :

- Variable d'environnement `GROQ_API_KEY`
- Base de données (table `SystemSetting` , clé `groq_api_key`) — **prioritaire**
- Panneau d'administration (sauvegarde en DB + runtime)

9.2 — Pipeline de génération de rapport

Le rapport généré par IA contient :

- **Titre** — Généré automatiquement
- **Synthèse narrative** — Résumé structuré en prose juridique
- **Points clés** — Liste extraite des éléments importants
- **Actions à entreprendre** — Tâches concrètes identifiées
- **Observations juridiques** — Notes légales pertinentes
- **Suggestions IA** — Typées : [LOI], [ARGUMENT], [DEFENSE], [OUVERTURE], [JURISPRUDENCE], [PREUVE]

9.3 — Intégration WhatsApp Cloud API

Le système est connecté via l'API WhatsApp Cloud (Meta). Variables d'environnement requises :

- `WHATSAPP_ACCESS_TOKEN` — Token d'accès Meta
- `WHATSAPP_PHONE_NUMBER_ID` — ID du numéro
- `WHATSAPP_VERIFY_TOKEN` — Token de vérification webhook

- `WHATSAPP_BUSINESS_ACCOUNT_ID` — ID du compte Business

9.4 — Pipeline WhatsApp

Lorsqu'un message WhatsApp est reçu :

- **Étape 1** — Détection IA : `detectHearingReportAI(text)` analyse si le message parle d'une audience (confiance ≥ 0.6)
- **Étape 2** — Extraction IA : `parseWhatsAppMessageAI(text, sender)` extrait date, client, référence, juridiction, chambre, avocat, adversaire, résumé professionnel, tâches
- **Étape 3** — Déduplication : vérification `whatsappMessageId` pour éviter les doublons
- **Étape 4** — Création HearingReport avec `source: "whatsapp"` et association à un utilisateur
- **Étape 5** — Confirmation envoyée via WhatsApp au sender

Si aucune clé IA n'est disponible, le système utilise le **fallback regex** (fonctionne uniquement pour les messages structurés).

SECTION 10

Internationalisation & Journal d'audit

Système de traduction et traçabilité complète des actions.

10.1 — Internationalisation (i18n)

FRANÇAIS (FR)	ENGLISH (EN)
Langue par défaut	Complètement supporté

- Système léger basé sur un dictionnaire clé-valeur dans `src/lib/i18n.ts`
- Fonction `t(key, locale)` pour la récupération des traductions
- Type `Locale = "fr" | "en" | "fon"` (Fon : support partiel)
- Locale stockée dans Zustand et persistée dans `localStorage`
- Fallback automatique sur le français si une clé n'est pas trouvée
- ~370 clés couvrant navigation, formulaires, messages, tableaux, module rôles (~65 clés dédiées)

10.2 — Journal d'audit

Toutes les actions utilisateur sont enregistrées avec métadonnées complètes (IP, User-Agent, timestamp, détails JSON).

ACTIONS AUDITÉES

> **Authentification :** login · register
> **Sessions :** create_session · delete_session · update_session
> **Audio :** upload_audio · transcribe · flash_record
> **Rapports :** generate_report · export_pdf · delete_report
> **Profil :** update_profile · change_password
> **Audiences :** create_hearing_report · update_hearing_report · delete_hearing_report
> **Rôles :** create_weekly_role · update_weekly_role · publish_weekly_role · delete_weekly_role
> **Admin :** admin_block · admin_unblock · admin_delete · admin_edit · admin_change_role · admin_approve · admin_reject · admin_update_llm · admin_reset_password

STRUCTURE D'UNE ENTRÉE D'AUDIT

```
{
  "id": "cuid...",
```

```
    "userId": "cuid...",
    "action": "create_session",
    "entity": "session",
    "entityId": "cuid...",
    "details": "{\"title\":\"Audience TPI\",\"clientName\":\"AHOUANDJINOU\"}",
    "ipAddress": "41.85.xxx.xxx",
    "userAgent": "Mozilla/5.0 ...",
    "createdAt": "2026-03-15T10:30:00Z"
  }
```